

FORGE

A Certificate-First Successor Architecture for Lean's grind

Concrete algorithms, exact prototypes,
and a staged implementation plan

The central proposal

Keep grind as the well-engineered search baseline. Add a goal-directed layer that can invent useful obligations, synthesize induction invariants, and request specialized certificates. Admit only checked consequences into the shared proof state.

What was executed	Four Python algorithm prototypes, exact certificate replay, and 260 passing automated tests.
What is proposed	A Lean tactic integrating these mechanisms with existing proof search and decision procedures.
What is not claimed	A compiled Lean implementation, a grind benchmark win, or a complete nonlinear/inductive prover.

14 September 2026

Source inspection targets the current manual and selected Lean v4.34.0 internals. Experimental Python versions and raw results accompany this article.

Abstract

A substantial improvement over Lean’s grind should expand the forms of reasoning available to automation, not merely add another rewrite loop. This article proposes *Forge*: a certificate-first architecture combining existing saturation with demand-driven backward reasoning, proof-producing nonlinear arithmetic, bounded induction and invariant synthesis, and specialized finite-domain solvers. The difficult engineering problem is controlled interaction: a nonlinear engine needs a bound that a recursive invariant can provide; an equality rewrite needs a side condition; a quantified theorem needs a term that is not yet present. *Forge* represents these needs as typed obligations, schedules their proofs, and exchanges only checked facts.

Four mechanisms are implemented in Python. An exact-replayed polynomial-cone search proves 60 of 60 constructed instances, versus 17 for a restricted dictionary. Bernstein subdivision certifies 21 positive-polynomial cases whose unsplit certificates all fail. An affine accumulator-invariant synthesizer solves 121 parameter instances. A finite ground Horn experiment reduces rule firings from 24,012 for full saturation, or 11,012 for an early-goal baseline, to 12 on a deliberately noisy instance. These are controlled algorithm experiments, not measurements of Lean or grind. The accompanying suite has 260 passing tests, including malformed certificates, cyclic derivations, differential polynomial checks, and intentional unknown outcomes. The article specifies mathematical contracts, Lean integration points, resource policies, realistic limitations, and the comparative evaluation required before claiming a production successor.

Contents

1	The objective: expand the search language, not just the portfolio	2
2	Start from the actual grind, not an obsolete caricature	2
3	Architecture: a typed obligation graph and a proved-fact ledger	3
4	Scheduling useful work without letting it explode	5
5	Demand-driven quantifiers and witnesses	6
6	Nonlinear arithmetic I: exact-replayed polynomial cones	8
7	Nonlinear arithmetic II: Bernstein box certificates	10
8	Induction and invariant synthesis: the largest capability jump	11
9	Other engines: use strong libraries, with explicit trust boundaries	13
10	Concrete Lean implementation plan and interfaces	15
11	What was actually implemented and measured	17
12	The experiment required before calling Forge a better Lean tactic	19
13	Development sequence and principal risks	20
14	Conclusion	20
A	Archive contents and reproduction	21
B	Checker contracts in compact mathematical form	22
C	Sources and version policy	22

1 The objective: expand the search language, not just the portfolio

1.1 What “more powerful” should mean

The target is a tactic that discharges significantly more ordinary mathematical and verification obligations without requiring the author to identify every missing intermediate lemma. There are three distinct dimensions: the kinds of proofs it can construct, the amount of guidance it needs, and its success rate under a fixed resource allowance. Improving the first does not automatically improve the third.

Forge’s proposed substantive additions are automatic induction with generalization and bounded invariant synthesis; stronger nonlinear inequality certificates; goal-directed creation of terms and obligations; and proof-aware scheduling across these processes. The tactic should also exploit specialized engines for Boolean and bit-vector problems, rather than force every problem through a general-purpose branch search.

A slogan such as “combine grind, aesop, and nlinearith” is insufficient. A sequential portfolio can try more terminal solvers, but cannot necessarily discover the intermediate assertion that makes one of them applicable. The main proposal is an *obligation broker*: producers state what they need, search proves those needs, and only then do producers contribute consequences. A simple portfolio is nevertheless an essential baseline against which the extra architecture must earn its complexity.

1.2 A precise but limited preservation claim

Let G_B be the problems solved by an unchanged grind invocation under a specified budget B . A cascade which first executes exactly that invocation, commits a successful result, and spends *additional* resources only on failure preserves G_B . This assumes identical imported declarations, options, initial state, and effective budget; even feature extraction before the baseline must not silently consume its reserved allowance.

It does *not* follow that a cascade preserves all successes under the same total wall-clock limit. Nor does modifying the initial lemma set preserve finite-budget behavior: logically helpful facts can make search slower. A practical interface should therefore distinguish a baseline-preserving extended mode from a fixed-budget competition mode. Recent work on learned interventions in grind independently advocates failure-triggered intervention because changing always-on heuristics can lose existing successes; Forge adopts that conservative principle, not a claim to have invented it [6].

A claim this article does not make

No actual Lean compiler or grind invocation was executed in the available environment. The included Python algorithms are executable and tested, while the full tactic architecture and Lean proof examples remain a design and uncompiled integration material. The experimental ablations must not be relabeled as grind results.

2 Start from the actual grind, not an obsolete caricature

The current manual describes cooperating congruence closure, propagation, E-matching, case analysis, and satellite solvers, with proof terms for asserted facts. It also explicitly recommends specialized SAT-based reasoning rather than grind for sufficiently large Boolean/bit-level combinatorial encodings [1].

The algebraic component is already considerably stronger than polynomial normalization: its documented algorithm uses Gröbner-style completion. The separate linear solver supports suitable ordered algebraic structures, not just a hand-written rational arithmetic fragment [2, 3]. Therefore “add a Gröbner basis” or “add linear real arithmetic” is not an adequate novelty claim.

The E-matching documentation already includes generation and instance bounds, value/size constraints, delayed guards, and attempts to prove guards by contradiction. Extensionality-related and introduction annotations also exist [4]. Forge must reuse those facilities rather than market them as new.

Selected v4.34.0 source inspection gives a useful integration boundary. The Action control layer in `Lean.Meta.Grind` supports progress, closure, and stuck states; its solver adapter processes newly propagated facts. Its continuation-based design already supports non-chronological backtracking and replayable script extraction. It also invokes model-based theory combination [5]. Merely adding backtracking, a fact store, or a model-combination label would not distinguish Forge.

Area	Reuse	Proposed additional capability
Ground reasoning	Existing equality and arithmetic cooperation	Requests for missing bounds, witnesses, and invariants, not just terminal solver calls.
Quantifiers	Existing E-matching and guards	Backward obligations and bounded typed term generation when useful instances lack trigger terms.
Algebra	Existing ring/field solver	Exact inequality cones, rational SOS reconstruction, and box certificates.
Recursive programs	Equational simplification and existing lemmas	Automatic recursor selection, dependent generalization, and proved invariant synthesis.
Finite problems	Existing specialized tactics	Verified translation and a resource-aware route to the appropriate solver.
User experience	Existing proof minimization	A compact explanation of the missing ingredient, with a replayable cross-solver proof.

Table 1: An extension agenda. The right column is a design objective, not a claim that every listed use case currently defeats grind.

3 Architecture: a typed obligation graph and a proved-fact ledger

3.1 Two kinds of objects that must never be confused

Forge maintains a graph of *obligations* and a ledger of *proved facts*. An obligation consists of a local telescope Γ , target P , relevant environment identity, and search metadata. It does not contain a fictitious proof of P . A proved fact contains an actual checked term p such that $\Gamma \vdash p : P$, together with its dependencies and provenance.

For example, an algebraic engine trying to clear a denominator may request $d \neq 0$. The request is not permission to rewrite a/d as though d were nonzero. Only a successfully reconstructed proof unlocks the rewrite. Likewise, a guessed induction invariant is an obligation; it is not inserted into the hypothesis set while proving itself.

A proof graph contains AND nodes for transformations requiring every child, OR nodes for alternative transformations, and edges from facts to consequences. Constructor reasoning on a

conjunction gives an AND node. Competing induction variables or witness expressions give OR alternatives. This structure closely matches Aesop’s existing proof-search machinery, which is a better starting point than a new handwritten search tree [7].

3.2 The ledger invariant

Write a fact record schematically as

$$F = (P, p, \Delta, E, \tau, \mu, \rho),$$

where P is the proposition, p its proof, Δ the assumptions it depends on, E the environment fingerprint, τ the local-telescope identity, μ the relevant metavariable assignment state, and ρ provenance and cost information. Sharing is permitted only if the proof can be instantiated in the destination context and checked there.

Dependency-set inclusion $\Delta \subseteq \Gamma$ is necessary but not sufficient. Free-variable identities, type-class instances, universe assignments, local definitions, and metavariables can invalidate a seemingly identical assertion. Cache entries should preferably store closed, telescope-abstracted theorem terms with explicit parameters rather than dangling local identifiers. Hashes are lookup accelerators, not proofs of syntactic or definitional equality.

Proposition 3.1 (Conditional soundness of the broker). *Suppose every committed transformation provides a Lean proof term of its claimed implication, all child goals are discharged, and every exported fact is well-typed in its destination context. Then every closed root obligation has a Lean proof under its original assumptions, independently of the heuristics used to select transformations.*

Proof. Induct over the finite acyclic proof graph after all selected alternatives have been fixed. Leaves are checked facts. At an internal node, substitute the checked child proofs into the checked transformation proof. The resulting term has the parent’s target type. No search ranking or external solver answer occurs as a logical premise. $\square$

This proposition states the intended Lean contract. It is not a formal verification of the Python implementation or a proof that a future implementation satisfies the contract.

3.3 Transactional state and branch-local learning

Each speculative branch receives an isolated saved tactic/metaprogramming state. Returning failure restores goal assignments, local instances, messages that should not survive, auxiliary declarations, and engine-local caches. In particular, assigning a shared metavariable in one alternative must not constrain a different alternative after rollback.

A fact derived using a split assumption H may be retained outside that branch only after discharge, for example as $H \rightarrow P$. A contradiction under $H_1, \dots, H_k$ can produce the clause $\neg H_1 \vee \dots \vee \neg H_k$ through an actual proof; it cannot be exported as an unconditional contradiction. Negative search results are cached as “this engine did not finish under these bounds,” never as mathematical negations.

The conservative first implementation should share only closed facts at tactic boundaries. Incremental sharing of partially solved dependent contexts is an optimization to add after the simpler version passes adversarial branch-isolation tests.

3.4 Three outcomes, not an overloaded Boolean

A producer returns a certificate candidate, UNKNOWN, or a candidate counterexample. The checker promotes a certificate to PROVED only after replay. A counterexample becomes REFUTED only when it genuinely refutes the original proposition under its original assumptions. A satisfying assignment to a weakened arithmetic abstraction, for instance, is merely a search diagnostic; it need not correspond to a Lean value satisfying dependent constraints.

The distinction is operationally important. A negative Bernstein coefficient is a loose lower bound, not a negative value of the polynomial. A failed LP reconstruction is not evidence of nonnegativity failure. An unsuccessful affine invariant template says nothing about richer templates or other proofs.

4 Scheduling useful work without letting it explode

4.1 A concrete phased policy

The default extended mode first runs the unchanged baseline. On failure, classify the remaining obligations by a small typed feature vector: recursive-call heads, target connective, participating algebraic structures, finite bit widths, nonlinear degree, and the number of unresolved side conditions. Classification proposes actions but never discards the baseline permanently.

The initial fallback sequence should be deliberately simple: inexpensive simplification and existing specialized tactics; bounded backward theorem application and witness construction; shallow induction candidates; small nonlinear certificate dictionaries; and then more expensive subdivision, superposition, or external certificate producers. Only obligations relevant to a stalled action are added in the initial mode.

A reasonable initial *experimental configuration*, not a measured optimum, is: at most 32 proposed backward lemmas per goal, 64 new witness terms per round, induction depth 2, one auxiliary lemma per induction branch, polynomial degree 4, 3,000 cone columns, and 4,096 subdivision nodes. The production user interface should expose these limits and the reason each was reached.

Listing 1: Scheduler pseudocode; not an existing Lean API.

```
solve(root, baselineBudget, extensionBudget):
  result = runIsolated(stockGrind, root, baselineBudget)
  if result.hasCheckedProof: return result
  queue = classifyAndSeed(root)
  while extensionBudget.remains and queue.notEmpty:
    action = selectByBenefitCostAndAge(queue)
    proposal = runIsolated(action, action.localBudget)
    match proposal:
      checkedProof  -> closeNodeAndPropagate()
      checkedFacts  -> commitFacts(); wakeSubscribers()
      obligations   -> addANDorORChildrenWithinBounds()
      unknown       -> recordBoundedFailure()
      modelCandidate-> validateOrKeepAsDiagnostic()
  return unknownWithFrontierSummary()
```

4.2 Benefit should include certificate cost

The ranking function can begin as a deterministic weighted score,

$$\text{score}(a) = \frac{\hat{p}_{\text{success}}(a) [1 + \text{demand}(a)]}{1 + \hat{t}_{\text{search}}(a) + \hat{t}_{\text{replay}}(a)} + \eta \text{age}(a).$$

Here demand counts blocked consumers of a result; it does not assert logical importance. The initial estimates can be coarse buckets updated from local observations. More sophisticated learning should be optional and trained on disjoint projects. A predictor is allowed to change priority, not correctness or the trusted checker.

Proof reconstruction can dominate search. The included Bernstein experiment already exhibits this: expanding certificate identities in the independent checker takes longer in aggregate than producing the certificates. A scheduler that measures only external solver time would optimize the wrong quantity.

4.3 Fairness and resource accounting

With K fixed strategies, restart each with budgets $1, 2, 4, \dots$ in deterministic rounds. If one strategy succeeds when given T work units, choose k minimal with $2^k \geq T$. The total allocation through round k is

$$K \sum_{j=0}^k 2^j < K 2^{k+1} < 4KT.$$

This elementary guarantee assumes comparable work units, bounded restart overhead, and a strategy whose successful behavior is preserved when its own budget grows. It is not a wall-clock theorem about competing native processes. For a growing strategy family, assign summable shares or explicitly bound the active set.

Track heartbeats, wall time, memory, generated clauses, certificate bytes, and polynomial coefficient bit length separately. Apply operating-system limits to external producers. A timeout should be interruptible without corrupting the Lean state. Global and per-goal quotas must prevent the multiplication of tiny per-action budgets into an unbounded search tree.

5 Demand-driven quantifiers and witnesses

5.1 Why waiting for ground terms is not always enough

E-matching reacts to available terms. A backward search can instead see that a theorem's conclusion resembles the desired assertion and propose the missing premises or typed instantiations. Forge uses both directions: forward saturation for cheap consequences, backward obligations when a useful conclusion remains unexplained.

Construct an index keyed by the elaborated head symbol of a theorem's conclusion, with coarse argument-type information. Match only after opening the theorem's telescope and instantiating universe parameters. A candidate application yields a proof-producing transformation with premise obligations; unresolved data arguments become typed witness holes. Premise order should prefer cheap propositions that constrain later witnesses, but dependency order in the telescope is mandatory.

For example, from $d \neq 0$, the goal $\exists z : \mathbb{R}, dz = c$ suggests $z = c/d$. The quotient must typecheck and the cancellation theorem must receive the nonzero proof. Blindly enumerating every term

in scope is unnecessary; the theorem conclusion supplies a strong syntactic constraint. This is a target use case, not an executed failing-grind example.

5.2 Bounded typed term generation

Witness generation has a finite grammar in each round. Its terminals are local variables, small numerals, and theorem-instantiated constants. Its productions are constructors and functions whose result types can unify with the required type. Each generated term is elaborated under the actual context. Ranking penalizes term size, unproved side conditions, new recursive heads, and repeated use of the same symbol.

Keep a size-indexed frontier; memoize modulo a validated typed normal form and, when available, proved equalities. Under binders, never reuse a term that depends on a variable absent from the destination context. Existential witnesses introduced from hypotheses retain their scoping dependencies. A seemingly convenient Skolem constant must not escape its telescope.

Trigger gaps can also be addressed by controlled term introduction. Before constructing $g(t)$ merely to activate a pattern, identify a theorem instance or backward proof obligation that explains why the term is relevant. Keep the original unfiltered saturation branch as a fallback: relevance pruning is not complete for arbitrary quantified logic, especially when an apparently unrelated contradiction could close the goal.

5.3 A finite prototype with an exact preservation theorem

The executable `horn.py` implements a much smaller problem: finite ground positive Horn rules

$$a_1 \wedge \cdots \wedge a_k \longrightarrow b.$$

A reverse index maps heads to rules. Starting from the requested atom g , add every rule with demanded head and recursively demand its body atoms. Run indexed semi-naive forward propagation only on the selected rules. A separate checker replays the resulting topological derivation from the externally supplied initial facts.

Proposition 5.1 (Ground demand slicing). *For a finite positive ground Horn program and fixed initial facts, the backward slice described above derives g if and only if the full program derives g in its least fixed point.*

Proof. Every selected rule belongs to the full program, giving one direction. For the converse, take a finite derivation of g . The slice includes its final rule, then every rule in a derivation of every required premise, recursively. Induction on derivation height shows that all necessary rules are selected. Cycles without a fact-supported finite derivation prove nothing in either program. $\square$

The reverse index costs linear time in the input size. Thus the prototype does not provide a sublinear cold-query theorem. A prebuilt index can make repeated goal-local queries much cheaper, and pruning reduces generated facts even when initial indexing still costs $O(N)$. This experiment does not model E-matching modulo equality, higher-order instantiation, negation, or Lean’s own relevance mechanisms.

6 Nonlinear arithmetic I: exact-replayed polynomial cones

6.1 The certificate language

Start with rational-coefficient polynomials over real variables. The context supplies

$$g_i(x) \geq 0 \quad (1 \leq i \leq m), \quad h_j(x) = 0 \quad (1 \leq j \leq r),$$

and the goal is $p(x) \geq 0$. A certificate consists of an exact identity

$$p = \sum_{k=1}^s c_k q_k^2 \prod_{i \in I_k} g_i + \sum_{j=1}^r a_j h_j, \quad c_k \in \mathbb{Q}_{\geq 0}, \quad q_k, a_j \in \mathbb{Q}[x]. \quad (1)$$

Repeated factors in I_k are allowed. The empty product is 1. The equality multipliers have arbitrary signs.

Proposition 6.1 (Cone certificate soundness). *An exact identity of the form (1) proves $p(x) \geq 0$ at every real point satisfying the supplied hypotheses.*

Proof. Each square, rational coefficient, and hypothesis product in the first sum is nonnegative. Each equality term vanishes. Substitute the hypotheses into the identity and add the nonnegative terms. $\square$

The tiny logical argument is independent of how difficult the identity was to find. A contradiction certificate uses $p = -1$. A strict goal can sometimes be handled by certifying $p - \varepsilon \geq 0$ for a rational $\varepsilon > 0$. Not every strict implication has a uniform rational margin, so this is only a sufficient technique; strict hypotheses require additional proof rules rather than being silently weakened in a completeness claim.

6.2 The implemented search algorithm

The producer first chooses a finite dictionary of possible nonnegative polynomials. The implemented dictionary contains monomial squares and, optionally, squares of sums or differences of two affine monomials. It multiplies them by products of at most k inequality hypotheses and rejects columns exceeding total degree d . Signed monomial multiples of equality hypotheses give the unrestricted ideal part.

Writing each column in the same monomial basis converts (1) to

$$A\lambda = b, \quad \lambda \geq 0.$$

The experiment minimizes $\mathbf{1}^T \lambda$ with SciPy's HiGHS interface. The interface solves the required linear equality-constrained, nonnegative-variable problem, but its numerical success flag is not a mathematical certificate [13]. First rationalize the proposed weights with bounded denominators and check the exact identity. If that fails, use the proposed nonzero support and solve its linear system over $\mathbb{Q}$ with SymPy. Reject negative reconstructed coefficients and replay again.

In an underdetermined exact reconstruction, the prototype sets remaining free parameters to zero. This may miss feasible nonnegative solutions. A production reconstructor should solve the residual exact feasibility problem or explore alternative supports. Numerical support selection, denominator limits, dictionary truncation, and coefficient growth are all sources of UNKNOWN, not unsoundness.

The independent replay routine uses only exact sparse polynomial arithmetic. It checks the ordered, externally supplied hypotheses, coefficient signs, indices, arities, and identity. It does

not invoke SciPy or SymPy. The search code cannot replace the target with an easier one inside the certificate. The shared polynomial library is tested against SymPy, but remains ordinary Python code, not a verified proof kernel.

6.3 A concrete mixed-theory certificate

Suppose

$$x + y = 1, \quad u = xy, \quad u \geq \frac{1}{3}.$$

The producer found the following certificate:

$$\begin{aligned} -1 &= 3(x - y)^2 + 12(u - \tfrac{1}{3}) \\ &\quad - 3(x + y + 1)(x + y - 1) - 12(u - xy). \end{aligned} \tag{2}$$

The last two terms vanish; the first two are nonnegative. Hence the hypotheses are inconsistent. Expanding the right side gives

$$3x^2 - 6xy + 3y^2 + 12u - 4 - 3(x^2 + 2xy + y^2 - 1) - 12u + 12xy = -1.$$

This exact identity is saved in `results/mixed_certificate.json`. It illustrates how a square inequality and a polynomial equality can jointly strengthen a bound. It is not advertised as a theorem that existing `nlinarith` cannot prove; indeed, a short conventional proof using the square is included as an uncompiled Lean example.

A second useful pattern is

$$xy - ab = (x - a)(y - b) + b(x - a) + a(y - b), \quad a, b \geq 0.$$

Given $x \geq a$ and $y \geq b$, the product of hypotheses provides the missing nonlinear consequence. A policy that admits only one inequality factor per dictionary column can miss this certificate.

6.4 What would make this stronger than the small prototype?

The first production stage should reuse `positivity`, `nlinarith`, and existing ring normalization. Mathlib’s linear-arithmetic infrastructure already separates search from proof reconstruction, an appropriate architectural precedent [8, 9]. The extension begins where those cheap calls stall.

Three concrete additions are worthwhile. First, seed squares from the actual syntax, differences between related arithmetic terms, and failed linear countermodels, rather than only monomials with coefficient ± 1 . Second, normalize equality consequences with the existing ring solver while retaining multiplier traces, so the inequality search works in a smaller quotient basis. Third, use an external semidefinite producer for a sparse Gram matrix when the small linear dictionary fails.

For the latter, write $p = z^T Q z$ or its hypothesis-weighted counterpart, where z is a monomial vector. A floating-point positive-semidefinite matrix is not enough. Reconstruct a rational symmetric Q and an exact factorization

$$Q = LDL^T, \quad D = \text{diag}(d_1, \dots, d_s), \quad d_i \geq 0.$$

Then $z^T Q z = \sum_i d_i ((L^T z)_i)^2$ reduces to (1). Zero pivots and permutations require explicit checking; a tolerance-based eigenvalue test is unacceptable. CVXPY is one possible modeling layer for the external proposal problem, not a trusted checker [14]. Exact rational SOS reconstruction is a separate established subject and should be treated as such [15].

This is not a claim that all nonnegative multivariate polynomials are SOS, or that a bounded cone search decides arbitrary nonlinear arithmetic. Even the globally nonnegative polynomial $(x - 2y)^2$ lies outside the prototype’s degree-two dictionary of monomial and unit-coefficient binomial squares. The test suite deliberately records UNKNOWN for that search.

6.5 Domain and coercion discipline

The initial Lean target should be $\mathbb{Q}$ and $\mathbb{R}$, followed by explicitly supported ordered fields. Rational coefficients do not embed into an arbitrary commutative ring with the required order properties. Natural subtraction, integer division, modular arithmetic, and bit-vector multiplication must not be translated as ordinary field operations.

Clearing a denominator in an inequality requires a sign proof or multiplication by a positive square, together with nonzero conditions. Cancelling factors requires their nonzeroness. Replacing $\sqrt{x}$ by y requires the appropriate domain facts and $y \geq 0$ in addition to $y^2 = x$. Casts from integers to reals preserve certain implications but can lose integrality information; a rational model is then only a model of the relaxation. These side conditions are exactly the obligations the broker must manage.

7 Nonlinear arithmetic II: Bernstein box certificates

7.1 A certificate that avoids numerical optimization

For a rational box $B = \prod_{j=1}^n [\ell_j, u_j]$ with $\ell_j < u_j$, substitute $x_j = \ell_j + (u_j - \ell_j)t_j$. Write the resulting polynomial in tensor Bernstein form,

$$p(\ell + (u - \ell)t) = \sum_{0 \leq i \leq d} \beta_i \prod_{j=1}^n \binom{d_j}{i_j} t_j^{i_j} (1 - t_j)^{d_j - i_j}. \quad (3)$$

Every basis term is nonnegative on $[0, 1]^n$, and the basis terms sum to 1. Thus

$$\min_i \beta_i \leq p(x) \leq \max_i \beta_i \quad (x \in B).$$

All coefficients can be rational. If the power-basis coefficient of t^a is c_a , a direct conversion is

$$\beta_i = \sum_{a \leq i} c_a \prod_{j=1}^n \frac{\binom{d_j}{a_j}}{\binom{d_j}{i_j}}. \quad (4)$$

Here d_j is at least the degree in variable j , and all summands have $a_j \leq i_j \leq d_j$, so denominators are nonzero. The identity follows by expressing each monomial in the Bernstein basis and multiplying the univariate identities.

7.2 Search and independent replay

At each box, compute the coefficients. If they are all nonnegative, emit a leaf certificate; for a strict bound, require all to be positive. Otherwise evaluate the center and corners exactly. A genuinely negative rational value gives a valid counterexample to a weak universal bound. If neither proof nor counterexample is available, bisect a widest coordinate at its rational midpoint and recurse on both children.

A split certificate stores only its coordinate, cut, and children. The checker derives the child boxes from the parent itself. It rejects a non-interior cut, a missing child, or a leaf with an invalid coefficient count. At a leaf it *expands* the Bernstein basis and checks (3); it does not trust the producer's conversion formula implementation.

The proof of a split uses $(x_j \leq c) \vee (c \leq x_j)$ and the two child proofs. Both children include the cut, so there is no coverage gap. A certificate cannot narrow the externally supplied root box. For a constrained semialgebraic domain, certifying an enclosing box is sufficient but may be too coarse; pruning an infeasible child requires a separate checked infeasibility proof.

Proposition 7.1 (Subdivision certificate soundness). *If every leaf has a valid identity (3) with nonnegative coefficients and every internal split covers its parent, the tree proves nonnegativity on the root box. Strictly positive coefficients at every leaf prove strict positivity.*

Proof. The Bernstein partition of unity proves the leaf claims. Induction over the finite tree and the coordinate split disjunction prove the parent claims, hence the root claim. $\square$

7.3 A sharp limitation: interior zeros

The example

$$p(x) = (x^2 - 2)^2, \quad x \in [1, 2],$$

is globally nonnegative, but a rational subdivision always has a leaf containing $\sqrt{2}$ strictly in its interior. All Bernstein basis functions are strictly positive there. If every coefficient on that leaf were nonnegative, their weighted sum could vanish only if every coefficient were zero, forcing the polynomial to vanish identically on that interval. It does not. Consequently, no finite rational-box tree using only this leaf criterion proves the bound.

This is a structural limitation, not merely a low depth setting. A square certificate proves the same assertion immediately. Conversely, box subdivision can certify positive polynomials that a small fixed square dictionary misses. The appropriate architecture is complementary producers, not a single universally preferred solver.

For a polynomial strictly positive on a compact box, a positive minimum exists. Under sufficiently fine subdivisions, the Bernstein coefficients approach local polynomial values uniformly: after affine rescaling, nonconstant coefficients are controlled by bounded derivatives times shrinking box widths. Hence exhaustive sufficiently fine subdivision eventually yields positive coefficients. This observation gives a useful strictly-positive regime, but says nothing about practical node counts or bounded-resource success.

7.4 Complexity and Lean implementation

A tensor certificate has $\prod_j (d_j + 1)$ coefficients per leaf. The dimensional growth is severe, even when total degree is modest. Production engineering should exploit variable sparsity, split only influential dimensions, use exact de Casteljau subdivision, share coefficient vectors, and choose between a sparse polynomial identity proof and a reflective checker.

A Lean implementation needs a theorem for the nonnegative partition of unity, correctness of the affine domain transformation, an exact polynomial normalization checker, and an inductive soundness theorem for the split tree. The current Python checker is a specification prototype for that work. Hostile serialized certificates additionally need parser limits on depth, coefficient size, allocation, and total nodes; the provided experiment works with in-process dataclasses, not an audited network format.

8 Induction and invariant synthesis: the largest capability jump

8.1 Induction is a transformation, not an aggressive case split

Repeatedly splitting an arbitrary list exposes only bounded prefixes. Structural induction adds an induction hypothesis that applies to a smaller recursive argument. Forge should search for this transformation explicitly instead of hoping saturation will simulate it. Existing equational lemmas remain useful for simplifying the resulting constructor cases.

Aesop’s documented design deliberately leaves induction to the user while automating the cases once induction has been selected [7]. Forge’s objective is to automate that selection and its most common missing ingredient: generalization. This is an integration and search objective, not a claim that automatic induction itself is new; rippling and related inductive-proof heuristics provide substantial prior art [12].

8.2 A concrete induction-candidate algorithm

Inspect elaborated recursive definitions occurring in the goal and retrieve their equational theorems. For every candidate structural argument, record which variables block reduction and which other arguments change in recursive calls. Rank candidates by: occurrence in blocked recursive calls, agreement between goal and context recursion, number of constructor cases, and predicted algebraic simplicity after unfolding. Use a small beam of candidates rather than committing to the first inductive variable in the local context.

Next compute a generalization set. If a recursive call updates an argument depending on a , include a . Close this set under dependencies in the local telescope: variables, hypotheses, and local instances whose types depend on reverted variables must be abstracted in a valid order. Construct the motive with these parameters universally quantified. This is where string-level manipulations of pretty-printed Lean goals become dangerous; use Lean’s elaborated expressions and recursor application machinery.

For each candidate, apply the actual recursor and create an AND node for its constructor cases. Simplify only with the chosen equations and bounded normalization. Try existing automation. If a case stalls because the induction hypothesis cannot match the recursive call, diagnose whether generalization is missing before launching larger search.

Well-founded or mutual recursion requires the corresponding recursor and its accessibility/decrease obligations, not an assumption that every argument is a list. The initial implementation should support a deliberately small structural fragment before extending to these cases. Dependent indexed types require motive construction that respects the indices; a cast-free erased first-order encoding is not a replacement.

8.3 An executed coefficient-synthesis example

Consider the integer-valued recursion

$$\begin{aligned}\text{run}([], a) &= a, \\ \text{run}(h :: t, a) &= \text{run}(t, ra + wh + c).\end{aligned}$$

Search the explicitly chosen affine template

$$\text{run}(xs, a) = Aa + B \text{sum}(xs) + C \text{length}(xs) + D. \quad (5)$$

The length is interpreted in the integer coefficient domain. The empty-list case requires $A = 1, D = 0$. In the constructor case, use the *generalized* induction hypothesis at $ra + wh + c$. Let $s = \text{sum}(t)$ and $\ell = \text{length}(t)$. The remaining identity is

$$A(ra + wh + c) + Bs + C\ell + D = Aa + B(h + s) + C(\ell + 1) + D.$$

Comparing coefficients yields

$$A = 1, \quad D = 0, \quad A(r - 1) = 0, \quad Aw - B = 0, \quad Ac - C = 0. \quad (6)$$

For $r = 1$, the solution is $A = 1, B = w, C = c, D = 0$. The executable producer solves these linear constraints over $\mathbb{Q}$; its independent checker rebuilds and verifies the universal base and step polynomial identities.

Proposition 8.1 (Synthesized accumulator invariant). *For all integers w, c, a and all finite lists of integers xs ,*

$$\text{run}(xs, a) = a + w \text{sum}(xs) + c \text{length}(xs) \quad \text{when } r = 1.$$

Proof. Induct on xs , universally generalizing a . The empty-list case is immediate. In the constructor case, apply the induction hypothesis at $a + wh + c$ and use the two defining equations for sum and length . The remaining equality is the polynomial identity verified above. $\square$

Without generalizing a , the induction hypothesis concerns only the old accumulator and does not directly rewrite the recursive call. The producer succeeded for all 121 pairs $(w, c) \in \{-5, \dots, 5\}^2$. For $r \in \{-2, -1, 0, 2, 3\}$, the same template is inconsistent with (6); those five runs return UNKNOWN. Recurrences with $r \neq 1$ admit other descriptions, but not this uniformly affine accumulator template for arbitrary list lengths.

8.4 From the toy template to useful lemma discovery

Production templates should be generated from typed summaries already present in the goal: length , sum , membership , mapped values, accumulator relations, and selected homomorphisms. For numeric folds, begin with affine combinations and bounded-degree polynomial templates. For list-building accumulators, use typed contexts such as

$$\text{reverseAux}(xs, a) = \text{reverse}(xs) ++ a.$$

The latter may require append associativity or a related auxiliary lemma. Candidate lemmas are generated by anti-unifying the stalled goal and a useful induction-hypothesis shape, then searching a bounded grammar of bridging contexts.

A candidate’s proof is a separate obligation. Do not prove lemma L using L itself, directly or through a cycle of other conjectures. Track a conjecture-dependency graph and admit only completed proofs. Ordinary recursively constructed Lean proofs can use their recursor-provided induction hypotheses; this is different from inserting an unproved global conjecture. Optional example generation is useful for rejecting bad templates, but passing examples never promotes a candidate to a theorem.

A practical first release should cap induction nesting at two, generalization alternatives at a small beam, and auxiliary-lemma nesting at one. Deeper configurations should be explicit. This makes failure informative: “the selected accumulator invariant requires an append lemma outside the current grammar” is better than an unexplained timeout.

9 Other engines: use strong libraries, with explicit trust boundaries

9.1 Equality saturation in small typed sublanguages

An equality-saturation engine is useful when choosing a normalization direction prematurely hides a good representation. The egg library provides rebuilding and e-class analyses suitable for such workloads [11]. This does not justify replacing Lean’s existing congruence closure wholesale.

The proposed adapter reifies only a small typed equational language: for example, fixed-domain algebraic expressions, list constructors and selected operations, or bit-vector expressions of a fixed width. Each rewrite rule names a Lean theorem and records substitutions. An extracted equality is replayed as a sequence or DAG of theorem applications, congruences,

symmetries, and transitivity steps. Typeclass dictionaries and universe instances must be retained or reconstructed with evidence.

A conditional rewrite becomes a pending edge with a premise obligation. It is merged only after the premise is proved in the current branch. An inequality fact produced by a cone or box checker can discharge that premise. Rebuilding invalidates representative-dependent lookup keys; caches should track a merge epoch and recheck substitutions when equalities change. A term’s first appearance is not the only event that can enable a new match.

This is a specialized alternative-representation tool, not a generic dependent-expression eraser. A large unconditional distributivity rule set can explode; use a term/node cap, a rewrite budget, and a proof-extraction cost model. If existing grind normalization solves the problem, do not pay for a second equality graph.

9.2 Superposition and higher-order proof search

Duper is an existing proof-producing saturation prover and an appropriate optional terminal fallback. Its documented interface accepts selected facts, including hypotheses and definitions; it also provides configurable portfolios and proof-script output [10]. The initial integration should call that documented interface rather than assume an unverified incremental clause-import API.

Forge can contribute an actual advantage here through premise selection. Retrieve candidate theorems by typed symbol overlap and backward conclusion matching, then expand only a bounded relevance neighborhood. Separate local facts from global candidates and keep the user’s explicit lemma list highest priority. Higher-order unification and reasoning about potentially empty types should be left to a competent proof-producing backend or Lean’s own typed transformations, not approximated away silently.

An external first-order prover can also be useful, but a returned proof in another calculus is an integration project: monomorphization, lambda lifting, choice, extensionality, and inhabitation assumptions all need reconstruction. Merely saying “use an SMT solver” does not supply that bridge.

9.3 Boolean and bit-vector specialization

For genuinely finite encodings, select `bv_decide` or a compatible proof-producing SAT route. This follows the manual’s own advice for difficult combinatorial encodings [1]. The new work is recognizing a suitable fragment, generating a *proved* translation, and routing the result back to the original goal.

The frontend must preserve bit widths, signedness, overflow, and coercions. An unbounded integer multiplied in a ring cannot be treated as a modular bit-vector product. Conversely, bit-blasting a 64-bit multiplication may generate a large circuit; structural recognizers should prefer algebraic identities when they offer a smaller proof.

A solver’s certificate is not the whole trust story. The inspected v4.34.0 native-evaluation infrastructure, used by native decision procedures including `bv_decide`, compiles and evaluates a closed Boolean expression and then introduces a fresh axiom asserting that it equals true. This differs from a kernel reduction proof and makes a blacklist containing only an older name such as `Lean.ofReduceBool inadequate` [16].

A strict mode should reject `sorryAx`, unapproved generated axioms, and any other dependencies outside the configured logical trust profile; it needs a kernel-checkable replay path. A faster mode may allow native evaluation, but must report that expanded trust. Audit the final theorem’s complete axiom dependencies instead of inferring trust from the tactic name. A

whitelist may include the ordinary logical axioms explicitly accepted by the project, such as choice, propositional extensionality, and quotient soundness; that policy is distinct from allowing arbitrary generated computation axioms. Independent Lean proof checking is also relevant to the broader validation workflow [17].

9.4 Transcendental and analytic goals

A realistic successor should not claim to decide arbitrary expressions involving exponentials, trigonometric functions, limits, or integrals. A concrete first analytic extension is proof-backed interval enclosure: apply a library theorem giving a rational bound or Taylor remainder on a verified domain, then send the resulting polynomial obligation to the cone or Bernstein engine.

For example, a claimed enclosure for $\exp(x)$ on $[a, b]$ must carry the actual remainder theorem and the domain proof. An external numerical interval without a Lean derivation is only a proposal. Such theorem adapters can be registered for recurring analytic patterns, but their implementation and library coverage are outside the executed prototype.

10 Concrete Lean implementation plan and interfaces

10.1 Build a conservative bridge before an invasive integration

The first usable Lean implementation should keep the engines separated by proof boundaries. Run an existing tactic; on failure, prove a candidate intermediate assertion in a fresh goal; assert its completed proof in the original goal; then retry the consumer. This incurs some repeated normalization but dramatically reduces state-sharing risk.

Only after that version works should `Forge.Bridge.GrindV434` adapt the pinned internal `Action/GoalM` interface. The inspected action source provides a solver-result path distinguishing no result, internal progress, propagated facts, and closure; propagated facts must enter the normal processing pipeline [5]. A production adapter must track source changes and preserve the expected continuation and rollback contracts.

Use the existing Aesop goal/rule machinery for backward and inductive search where practical. Use Mathlib’s tactic frontends as terminal actions first. Specialize or lower-level-integrate them only when profiles identify repeated work as a serious bottleneck. Redesigning every established engine at once would delay the very experiment needed to justify Forge.

Table 2: Concrete dependencies and their intended role. Proposed adapters are not asserted to exist already.

Library / component	Use	Integration restriction
Lean core grind	Ground saturation and existing theory combination	Preserve a stock invocation; isolate a version-pinned internal bridge.
Lean metaprogramming	Telescope abstraction, recursor application, local proof goals	Manipulate elaborated expressions; never search by pretty-printed text alone.
Mathlib simp, omega, ring, field_simp, norm_num	Cheap normalization, arithmetic, and certificate identities	Guard domain-changing transformations and nonzero conditions.
Mathlib linarith, nlinarith, positivity	Cheap inequalities and replay support	Baselines and components, not newly invented algorithms.

Library / component	Use	Integration restriction
Aesop	AND–OR search, rules, normalization	Add bounded induction/witness producers; do not duplicate its full scheduler casually.
Duper	Selected-premise saturation fallback	Start with its public terminal tactic interface.
egg	Alternative expression forms in a typed fragment	Replay every extracted equality; no unchecked dependent erasure.
bv_decide / SAT route	Finite Boolean and bit-vector goals	Prove translation; audit the actual replay/evaluation trust profile.
SciPy / HiGHS	LP proposal for a finite cone dictionary	Numerical output is untrusted and requires exact reconstruction.
SymPy	Exact small linear systems and candidate algebra	Search utility only; independent replay is mandatory.
CVXPY / SDP producer	Future sparse Gram-matrix proposals	Rational matrix identity and PSD factorization must be checked.

10.2 Proposed module structure

The implementation should have small explicit boundaries rather than one giant metaprogram:

```

Forge/Frontend.lean      -- syntax, profiles, user diagnostics
Forge/Search/ObligationGraph.lean -- AND/OR nodes and scoped dependencies
Forge/Search/Induction.lean -- recursors, generalization, templates
Forge/Search/Witness.lean -- typed backward application/terms
Forge/Cert/Polynomial.lean -- exact representation and semantics
Forge/Cert/Cone.lean    -- identity + positivity replay
Forge/Cert/Bernstein.lean -- box leaves, split coverage, replay
Forge/Bridge/GrindV434.lean -- isolated internal-version adapter
Forge/Bridge/Mathlib.lean -- public existing-tactic actions
Forge/Trace.lean        -- proof DAG, minimization, replay

```

These are proposed paths. The archive contains Python implementations and illustrative Lean proof recipes, not this completed module tree.

The central interface should distinguish an engine’s *candidate* from a checked result:

Listing 2: Conceptual interface, with deliberately abstract names.

```

Engine.propose(problem, budget) ->
  CertificateCandidate | ObligationPlan | Unknown | ModelCandidate

Checker.replay(originalProblem, certificate) ->
  CheckedProof | Rejected

Broker.commit(chcheckedProof, destinationContext) ->
  ScopedFact | ContextMismatch

```

A `CheckedProof` must be constructed through a validating path. Giving an arbitrary `Expr` a suggestive structure name does not make it a proof. Before commitment, verify its target, assumptions, typeclass arguments, unresolved metavariables, and scope. Kernel validation of the final reconstructed declaration remains mandatory even when intermediate metaprogramming checks have succeeded.

10.3 A proposed user interface

The following commands are design sketches, not available syntax in this archive:

```
by forge
by forge [localLemma, usefulDefinition]
by forge (profile := extended) (inductionDepth := 2)
by forge? -- print a reduced ordinary proof script
by forge replay "proof.forgecert" -- no external search during replay
```

Profiles should make important choices explicit: *fast* for low-latency existing automation plus a small extension budget; *extended* for a preserved baseline allowance plus bounded new searches; and *fixedBudget* for a total-limit portfolio experiment. An orthogonal trust option specifies strict kernel replay or an explicitly approved native-evaluation profile.

Useful diagnostics identify a frontier, not a thousand low-level events: “induction on xs tried; accumulator a generalized; step case needs $d \neq 0$; certificate search exhausted degree 4; no counterexample verified.” When a proof succeeds, minimize theorem dependencies and certificate support, then replay the minimized script from a clean initial state. Removing a branch can also remove facts on which a supposedly redundant solver step depended; replay validation is essential.

11 What was actually implemented and measured

11.1 Scope, environment, and reproducibility

The executed artifact contains four producers and their checkers: a finite polynomial-cone LP search, exact Bernstein subdivision, affine fold-invariant synthesis, and demand-sliced finite ground Horn propagation. Shared sparse polynomial arithmetic uses Python Fraction coefficients and rejects floating-point coefficient input. Monomials are enumerated by weak compositions rather than a wasteful enclosing Cartesian grid.

The recorded environment is Python 3.13.5, NumPy 2.3.5, SciPy 1.17.0, SymPy 1.14.0, and pytest 9.0.2. The selected official Lean source was v4.34.0; no Lean executable was installed, and obtaining a compiler binary was unsuccessful. Consequently no Lean scripts, tactic integration, kernel-checking times, or actual grind baselines were run.

All workloads are deterministic and intentionally mechanism-focused. The timing columns are single warmed runs, with imports excluded and a small LP warmup. They are reproducibility observations, not statistically meaningful comparative speed claims. The experiment runner writes individual rows, aggregate JSON, environment information, and example certificates. The test log records **260 passing automated test cases**.

Experiment	Restricted baseline	Extended mechanism
Polynomial cones, 60 cases	17 certificates, 43 unknown	60 certificates, 0 unknown
Positive boxes, 21 cases	0 unsplit certificates	21 subdivision certificates
Affine invariants, 121 cases	No no-synthesis baseline measured	121 synthesized and replayed
Out-of-template recurrences	Not a proof/disproof comparison	5 honest unknown outcomes
Noisy Horn program	24,012 full-closure firings; 11,012 early-goal firings	12 demand-sliced firings

Table 3: Executed Python results. None of the restricted baselines is Lean’s grind.

11.2 Cone workload and ablation

The 60 cases comprise 25 positive product-bound implications, 15 binomial-square expressions, 6 mixed equality/inequality contradictions, 9 diagonal-square expressions, and 5 ideal consequences. Both configurations receive the same problems and total degree 2. The restricted configuration allows monomial squares and at most one inequality factor. The enriched configuration additionally allows the affine binomial squares and products of two inequalities.

Both use the same LP backend and exact replay. The restriction therefore isolates the effect of the candidate language, not an implementation-language or solver difference. The enriched producer takes about 0.197 seconds in aggregate and its separately timed replay about 0.008 seconds; the restricted producer takes about 0.074 seconds. The richer language solves more but is not free.

This suite is deliberately chosen to exercise the added dictionary entries. It demonstrates functioning mechanisms and regression coverage, not generalization to a theorem corpus. Additional tests reject negative certificate coefficients, invalid assumption indices, float weights, changed targets, missing inequalities, corrupted equality multipliers, and reordered hypotheses. True statements outside the chosen dictionary are expected to remain unknown.

11.3 Bernstein workload and counterexamples

Eighteen univariate instances have the form

$$(x - k/7)^2 + 1/d, \quad k = 1, \dots, 6, \quad d \in \{10, 100, 1000\}, \quad x \in [0, 1].$$

Three bivariate instances use $(x - 2/7)^2 + (y - 3/7)^2 + 1/d$ on $[0, 1]^2$. Every unsplit strict certificate fails; subdivision proves all 21, using 2–21 leaves. Aggregate search takes about 0.049 seconds and independent replay about 0.083 seconds.

The tests also check actual negative-point witnesses, boundary zeros under strict versus weak goals, invalid split geometry, missing children, incorrect coefficient vectors, changed polynomials, and the irrational-interior-zero limitation. Fifteen randomized bivariate power-to-Bernstein conversions are verified by a different basis-expansion algorithm after adding an exact positive shift.

11.4 Invariant and polynomial differential tests

For each of the 121 coefficient pairs, synthesis returns the expected affine invariant and exact replay validates the two symbolic proof obligations. The test suite additionally runs four deterministic random-list checks per pair, giving 484 concrete recurrence checks. These are bug detectors supplementing the symbolic identity checks; neither constitutes Lean kernel verification.

Thirty seeded sparse-polynomial tests compare addition, multiplication, and evaluation with SymPy. The suite tests monomial enumeration size against $\binom{n+d}{d}$, coefficient-type validation, malformed exponents, and arity mismatches. These checks reduce the chance of a shared arithmetic bug but do not eliminate it. A formally proved Lean checker remains a separate deliverable.

11.5 Horn workload: structural savings and their limits

A useful chain has length 12. The largest noise set contains 1,000 unrelated chains of length 24, each with an initial fact. Full closure fires every one of the 24,012 rules. An indexed baseline that

stops as soon as the goal is known still fires 11,012 rules under the specified interleaved queue order. Backward slicing selects just the 12 useful rules.

Proof minimization reduces even the full-closure derivation to 12 proof steps. Thus proof size alone would conceal the large search-work difference. Conversely, the cold reverse index still scans all input rules, and the demand implementation still loads the initial fact set. This is not a claim of sublinear end-to-end cold performance. Raw timings contain environmental/allocator noise, including a slower early-goal run despite fewer firings; no speedup factor is inferred from them.

Forty randomized finite Horn programs compare full and sliced success and replay their derivations. Additional tests reject unsupported cycles and malformed rule references. The exact preservation theorem applies only to this finite positive fragment. It cannot be transferred unchanged to arbitrary dependent theorem proving or to contradiction search with omitted premises.

12 The experiment required before calling Forge a better Lean tactic

12.1 Comparators and datasets

A meaningful Lean evaluation needs at least five comparators: unchanged `grind`; `grind` with reasonable supplied lemmas and documented tuning; `Aesop`; a simple terminal portfolio of existing tactics; and the integrated Forge design. Where `Duper` or `bv_decide` is appropriate, include it rather than select only tasks that favor a general-purpose method.

Use separate strata for ordinary library goals, arithmetic, recursive data structures, verification conditions, quantified relational reasoning, and finite combinatorial problems. Split by project or theorem family, not random nearly identical instances. Add unseen custom recursive definitions to prevent an “induction benchmark” from being solved entirely by memorized library lemmas. Preserve source order and prevent the target theorem from appearing in its own retrieval set.

Report the same complete environment: Lean and dependency commits, imported modules, compilation state, local options, theorem arguments, allowed hints, and trust policy. A `grind` call that is denied a lemma which Forge receives is not a fair measure of search capability. One can separately measure *automatic lemma retrieval*, but must label that comparison correctly.

12.2 Metrics and necessary ablations

Measure solved goals, unique wins and regressions, search and reconstruction time, cold-start and warmed time, peak memory, certificate bytes, final proof size, and trusted axiom dependencies. Separate timeout, memory exhaustion, unsupported fragment, certificate rejection, and actual verified counterexample. Repeat timing runs and report distributions rather than one fastest run.

Ablate the broker, induction generalization, invariant synthesis, new nonlinear atoms, subdivision, premise selection, learned ranking, and internal incremental sharing. In particular, compare a cheap existing-tactic portfolio plus manually supplied intermediate assertions with the same portfolio plus Forge’s assertion discovery. This isolates whether the difficult new component really contributes.

An appropriate acceptance gate is a measured, reproducible gain over the strongest reasonable simple baseline on held-out relevant strata, together with a regression budget and audited trust profile. It would be premature to choose an impressive percentage before observing the Lean results. The present Python suite establishes feasibility of components, not that acceptance gate.

13 Development sequence and principal risks

13.1 Milestones with executable completion criteria

Milestone 1: proof boundary and reproducibility. Implement the conservative Lean broker, unchanged baseline mode, scoped fact import, proof tracing, and a pinned benchmark runner. Completion requires clean rollback on failures and clean-process replay of every accepted result.

Milestone 2: certified nonlinear core. Port the exact polynomial representation, prove cone replay soundness, and reconstruct the included certificates in Lean. Add degree/size caps and malformed-input tests. Completion requires no external process during replay and axiom audits of every example.

Milestone 3: induction and generalization. Support a small structural class such as lists and naturals; implement telescope-correct generalization and affine accumulator templates. Completion requires unseen folds with parameterized updates, failure diagnostics, and rejection of circular conjecture proofs.

Milestone 4: box certificates and demand search. Prove the Bernstein tree checker and integrate bound requests. Add backward application and typed witnesses, then demonstrate a proof where an intermediate result from one mechanism enables another. Completion requires cross-solver ablations, not just a larger list of terminal successes.

Milestone 5: selective deep integration. Profile repeated normalization and state translation. Only then introduce the pinned internal Grind adapter, specialized equality saturation, or richer external certificates. Completion requires compatibility tests and a measurable benefit sufficient to justify maintenance.

These are engineering stages, not a promise of a particular delivery duration. The most expensive tasks are likely dependent-state integration, proof-term performance, general invariant discovery, and a reliable benchmark corpus, rather than the small LP search demonstrated here.

13.2 Failure modes that deserve explicit tests

The largest correctness risk is scope leakage: a fact depending on a split, existential witness, or metavariable assignment is reused elsewhere. The largest search risk is speculative obligation explosion. The largest performance risk is accepting a numerically easy certificate whose exact proof is enormous. The largest evaluation risk is conflating manually enriched contexts or synthetic dictionary examples with real tactic improvement.

Version fragility is another practical issue: the current manual is a moving reference, and internal metaprogramming interfaces are not a stable interchange protocol. Keep public-tactic adapters as a fallback, pin complete dependency sets, and record the exact engine configuration in replay artifacts.

Finally, external solver access should be an explicit policy choice. Lean terms and hypotheses may contain proprietary material. Local external processes need sandboxing; remote solvers require user authorization and data-minimization policy. The present prototype runs entirely locally after its dependencies are installed and sends no problem data to a service.

14 Conclusion

Forge’s strongest prospect is not a cleverer universal rewrite loop. It is the ability to discover and prove the intermediate assertions that let existing engines cooperate: generalized induc-

tion hypotheses, algebraic inequalities, domain bounds, side conditions, and witnesses. Each assertion must cross a proof boundary before it becomes available to another engine.

The executed work demonstrates exact nonlinear certificates, complementary box proofs, a real but narrow invariant synthesizer, and demand-sliced derivations with checkable provenance. It also demonstrates why realistic limitations matter: richer dictionaries cost more, box certificates can fail on nonnegative interior-zero polynomials, and reconstruction can dominate search.

The recommended implementation path is therefore conservative and concrete: retain grind, build a scoped obligation broker, port the small certificate checkers, automate induction generalization, and only then deepen incremental cooperation. This design plausibly expands the class of proofs that a single Lean tactic can construct. Establishing a significantly better production tactic requires the explicit Lean implementation and comparative evaluation described above; the archive does not pretend those have already been performed.

A Archive contents and reproduction

article/forge.tex	This article's LaTeX source
article/forge.pdf	Rendered article
prototype/poly.py	Exact sparse polynomials
prototype/cone.py	LP producer and exact cone replay
prototype/bernstein.py	Subdivision producer and tree replay
prototype/invariant.py	Affine coefficient synthesis/replay
prototype/horn.py	Demand slicing and derivation replay
prototype/tests/test_algorithms.py	
prototype/experiments.py	Deterministic workload generator
results/*.csv	Individual experiment observations
results/summary.json	Aggregated results
results/environment.json	Executed Python versions
results/*certificate.json	Example exact certificates
results/pytest.txt	Actual pytest log
lean/ReplayExamples.lean	UNCOMPILED conventional proof recipes
lean/INTEGRATION.md	Lean status and integration boundary
README.md	Build and execution instructions
requirements.txt	Versions used for Python experiments
Makefile	Test, experiment, and PDF targets

From the archive root, use:

```
python -m pip install -r requirements.txt
cd prototype
python -m pytest -q
OPENBLAS_NUM_THREADS=1 OMP_NUM_THREADS=1 python experiments.py
cd ../article
pdflatex -interaction=nonstopmode -halt-on-error forge.tex
pdflatex -interaction=nonstopmode -halt-on-error forge.tex
```

The supplied result files describe the recorded run; rerunning experiments overwrites the local results and may change timing columns. The article uses fixed rounded observations, not automatically refreshed measurements. There is no bundled Lean compiler, Mathlib checkout, font file, or external solver binary. The Lean recipes are intended to be checked in an independently installed, matching Mathlib project.

B Checker contracts in compact mathematical form

Cone. Input is $(p, (g_i), (h_j), C)$. Accept only if all references in C are valid, coefficients claimed nonnegative are exact nonnegative rationals, and the expanded polynomial identity is exactly p . The conclusion is conditional nonnegativity under the externally supplied hypotheses.

Bernstein. Input is (p, B, T, strict) . Accept only if every split is a full cover derived from B and every leaf exactly expands to the appropriate affine transform of p , with coefficient signs satisfying the requested strictness. A negative point is a separate certificate with independently checked domain membership and evaluation.

Invariant. Input is (r, w, c, A, B, C, D) . Accept only if the base and constructor-step polynomial residuals vanish. The mathematical bridge to all finite lists is structural induction with the accumulator generalized; Lean must replay that bridge, not just the arithmetic residuals.

Horn. Input is the original ground rules, initial facts, goal atom, and a topologically ordered derivation. Every step must name an original rule whose premises are already justified. The goal must be justified at the end. No cyclic self-support is accepted.

C Sources and version policy

The bibliography points to primary documentation, source repositories, and original papers. Public manual and generated-doc URLs are mutable; the selected Grind action source is pinned to v4.34.0. The prototype version manifest records what actually ran, rather than substituting current documentation versions for installed package versions. Source inspection does not imply that a library integration was compiled or benchmarked.

References

- [1] Lean project. *The Lean Language Reference: The grind tactic*. Accessed 14 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [2] Lean project. *Algebraic Solver (Commutative Rings, Fields)*. Current reference manual, accessed 14 September 2026. https://lean-lang.org/doc/reference/latest/The--grind--tactic/Algebraic-Solver-_LPAR_Commutative-Rings___-Fields_RPAR_/.
- [3] Lean project. *Linear Arithmetic Solver*. Current reference manual. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Linear-Arithmetic-Solver/>.
- [4] Lean project. *E-matching*. Current reference manual. https://lean-lang.org/doc/reference/latest/The--grind--tactic/E__matching/.
- [5] Lean contributors. *Lean.Meta.Tactic.Grind.Action*, source at tag v4.34.0. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Tactic/Grind/Action.lean>.
- [6] Evan Wang, Simon Chess, Sophie Szeto, and Theodore Meek. *Learned Interventions in Lean 4 grind*. arXiv:2607.22972, 2026. <https://arxiv.org/abs/2607.22972>.
- [7] Aesop contributors. *Aesop: Automated Extensible Search for Obvious Proofs*. Repository documentation. <https://github.com/leanprover-community/aesop/blob/master/README.md>.
- [8] Mathlib contributors. *Mathlib.Tactic.Linarith.Frontend*. Generated source documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Linarith/Frontend.html.
- [9] Mathlib contributors. *Mathlib.Tactic.Positivity.Basic*. Generated source documentation. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/Positivity/Basic.html.

- [10] Duper contributors. *Duper*. Repository documentation. <https://github.com/leanprover-community/duper/blob/main/README.md>.
- [11] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. *egg: Fast and Extensible Equality Saturation*. POPL 2021. DOI:10.1145/3434304. <https://arxiv.org/abs/2004.03082>.
- [12] Alan Bundy, Andrew Stevens, Frank van Harmelen, Andrew Ireland, and Alan Smaill. *Rippling: A heuristic for guiding inductive proofs*. Artificial Intelligence, 62(2):185–253, 1993. <https://www.sciencedirect.com/science/article/pii/000437029390079Q>.
- [13] SciPy developers. *scipy.optimize.linprog, method='highs'*. API reference. <https://docs.scipy.org/doc/scipy/reference/optimize.linprog-highs.html>. Prototype version: SciPy 1.17.0.
- [14] Steven Diamond and Stephen Boyd. *CVXPY: A Python-Embedded Modeling Language for Convex Optimization*. arXiv:1603.00943, 2016. <https://arxiv.org/abs/1603.00943>.
- [15] Victor Magron, Mohab Safey El Din, and Trung-Hieu Vu. *Sum of Squares Decompositions of Polynomials over their Gradient Ideals with Rational Coefficients*. arXiv:2107.11825, 2021. <https://arxiv.org/abs/2107.11825>.
- [16] Lean contributors. *Lean.Meta.Native*, source at tag v4.34.0. In particular, `nativeEqTrue` and its generated-axiom path. <https://github.com/leanprover/lean4/blob/v4.34.0/src/Lean/Meta/Native.lean>.
- [17] Mario Carneiro. *Lean4Lean: Towards a Verified Typechecker for Lean, in Lean*. arXiv:2403.14064, 2024. <https://arxiv.org/abs/2403.14064>.